

Day - 4 - Memory & Mapping

▼ Memory Mapping in ARM Cortex | Flash | RAM | Stack | Heap

1. Introduction to Memory Mapping

- **The Concept:** Processors themselves do not have built-in memory; it must be interfaced externally. This cannot be done blindly; memory mapping is the systematic and scientific design used to integrate memory with the processor.
- **Memory Limit:** Because ARM Cortex processors are 32-bit, they can interface with a maximum of **4 GB of total memory**.

2. The 4 GB Memory Map Layout

The 4 GB address space is divided into specific, standardized segments:

- **Code Memory (0.5 GB):**
 - Maps to Flash memory or EEPROM.
 - Address range starts from `0x00000000` to `0x1FFFFFFF`.
- **SRAM (0.5 GB):**
 - Static RAM is used to hold volatile data.
 - It contains dynamic memory allocations, specifically the **stack and the heap**.
 - Address range starts from `0x20000000` to `0x3FFFFFFF`.
- **Peripherals (0.5 GB):**
 - Dedicated space for standard built-in microcontroller peripherals such as GPIO, SPI, I2C, and CAN.
- **External Memory (1 GB):**
 - Reserved for interfacing external RAM or ROM.

- Used when the internal memory is insufficient for the application (e.g., an embedded system in a car that needs to record live video).
- **External Devices (1 GB):**
 - Used to interface extra external I/O devices that are not part of the regular built-in peripherals, such as USB interfaces, external motor drivers, or external CAN devices.
- **System Level Memory:**
 - Ranges from `0xE0000000` to `0xFFFFFFFF`.
 - This section is split into two distinct parts:
 1. **Private Peripherals:** Dedicated to components strictly related to the core processor, such as the **NVIC** (Nested Vector Interrupt Controller), System Control Block, Data Watch points, and ETM (Embedded Trace Macrocell).
 2. **Vendor Specific:** A reserved space allowing microcontroller manufacturers to map their own custom circuitry or proprietary peripherals.

Note: Developers must strictly follow these memory addresses. Violating these mapped regions will result in memory faults, memory aborts, or segmentation faults.

3. Bit Banding

- **Definition:** Bit banding is an optional feature that provides **atomic access to a single bit of memory**, allowing programmers to easily perform bitwise operations.
- **The Advantage:** It allows the ARM processor to manipulate (set or clear) a single bit using a single load and store instruction **without needing a complex read-modify-write cycle**. This is highly efficient for microcontrollers, which frequently rely on bit-level control for peripherals.
- **Memory Allocation:** The architecture allocates exactly **1 MB of memory** specifically for bit banding operations. Because it targets 32-bit lengths, this 1 MB area provides exactly **32 Megabits** (1 MB×32 bits) of distinct bit-level manipulation space.

▼ Memory Mapped IO | IO Mapped IO

1. Memory Mapped I/O

- **Definition:** In this design, the processor treats I/O devices (peripherals like SPI, I2C, GPIO, DAC, ADC) exactly like any other part of the main memory.
- **Memory Structure:** The peripherals are efficiently mapped into the main system memory alongside the RAM and ROM.
- **Addressing:** Each peripheral is assigned its own unique address within the main memory space. Depending on the processor's architecture, these addresses are typically **16-bit or 32-bit**.
- **Data Transfer:** Because the peripherals are considered part of the main memory, moving data to or from a peripheral is treated as a standard memory-to-memory transfer. It relies on the processor's regular data transfer instructions, making operations straightforward and efficient.

2. I/O Mapped I/O

- **Definition:** Also referred to simply as "Mapped I/O", this design physically and logically separates the main memory from the peripheral memory.
- **Memory Structure:** Main memory (containing RAM and ROM) operates independently from the space where peripherals (like timers, UART, SPI) are mapped.
- **Addressing & Ports:** The main memory might use a separate 16-bit address, while the peripherals are typically allocated a smaller **8-bit address**. These separate 8-bit addresses assigned to the I/O devices are referred to as **"ports"**.
- **Capacity:** By using an 8-bit address, the system can support and allocate a maximum of **256 different peripherals**.
- **Control Signal (IO/M):** Because there are two distinct memory spaces, the system utilizes a dedicated control signal (**IO/M** bar) to tell the processor which space it is interacting with:
 - **If IO/M = 0:** The processor recognizes this as a **main memory operation**.

- If IO/M = 1: The processor recognizes this as an I/O (peripheral) operation.

▼ Memory Organization in C | Heap | Stack | Data | Code

1. Overview of Memory Layout

When a C program is compiled, the compiler allocates different parts of memory for different types of variables and functions. The memory layout is broadly divided into two main categories: **Static Memory Allocation** and **Dynamic Memory Allocation**.

2. Static Memory Allocation

Static memory is allocated when the main program initializes and starts executing. It consists of two primary segments:

- **Code Segment (Text Segment):**
 - This segment stores the completely compiled machine code, including executable files, hex files, and binary files.
 - It operates exclusively as **read-only memory**.
- **Data Segment:** This is subdivided into two parts:
 - **Initialized Data Segment:** Stores variables that are explicitly declared and assigned a value other than zero (e.g., `int a = 10;`). These variables retain their assigned values throughout the entire program execution.
 - **Uninitialized Data Segment (BSS):** BSS stands for "**Block Started by Symbol**". It stores global variables and static variables that have not been assigned a specific value. By default, the system automatically assigns these variables a value of zero or null.

3. Dynamic Memory Allocation

Dynamic memory allocation refers to memory that is allocated during the runtime of the program. It consists of two segments that grow in opposite directions:

- **Stack Segment:**

- **Mechanism:** Operates on a **LIFO (Last-In-First-Out)** or First-In-Last-Out principle.
- **Growth:** The stack specifically **grows downwards** in memory.
- **Use Cases:** It is used heavily during **function calls** to store return addresses, local variables, and parameters. It is also essential for **interrupts** (for context storing and restoring) and during **recursive function calls**.

- **Heap Segment:**

- **Growth:** The heap **grows upwards** in memory.
- **Use Cases:** It is exclusively used for manual runtime memory allocation using pointer functions.
- **Functions:** It utilizes `malloc` (memory allocation), `calloc` (contiguous memory allocation), and `realloc` (reallocation) to assign memory blocks. To release this memory, the `free` function is used for deallocation.

4. Key Concept: Segmentation Fault

A **segmentation fault** is a critical error that occurs when a program violates memory layout rules or memory permissions. It happens if the code attempts to overwrite a part of the memory layout that is already defined and allocated for a different segment.